

EXPERIMENT 4

Customer Churn Prediction using Logistic Regression

1. Problem Definition

The objective of this experiment is to develop a model that predicts whether a customer will churn (leave a subscription-based service).

Churn is a binary variable:

- 1 → Customer leaves
- 0 → Customer stays

Since the output variable is binary, this is a Binary Classification Problem.

The goal is to learn:

$$P(\text{Churn}=1)=f(\text{Customer Features})$$

2. Data Understanding

The dataset contains 200 customer records.

Each row represents one customer.

The dataset includes 11 columns:

- CustomerID
- Gender
- Age
- Tenure_Months
- SubscriptionType
- MonthlyCharges
- TotalCharges

- PaymentMethod
- SupportTickets
- ContractType
- Churn

Target Variable

- Churn

Feature Variables

All remaining columns except Churn.

There are no missing values in the dataset.

3. Preprocessing (Encoding)

(a) Handling Missing Values

The dataset was checked for missing values and none were found.

`df = df.dropna()` was used as a safety step.

(b) Encoding Categorical Variables

Two encoding techniques were applied:

1. Label Encoding for object-type columns
2. One-Hot Encoding using:

`df_encoded = pd.get_dummies(df, drop_first = True)`

This avoids multicollinearity and converts categorical variables into numeric format suitable for Logistic Regression.

4. Feature–Target Split

The dataset was divided into:

- `y = df_encoded['Churn']`
- `X = df_encoded.drop('Churn', axis=1)`

This separates input features from the output variable.

5. Train-Test Split

The dataset was split into:

- 80% Training Data
- 20% Testing Data

Using:

`train_test_split(test_size=0.2, random_state=42)`

This ensures reproducibility.

Choose the Classification Model

The model used is Logistic Regression:

$$P(\text{Churn} = 1) = 1/(1 + e^{-z})$$

Where:

$$z = b_0 + b_1x_1 + b_2x_2 + \dots$$

The model outputs probability between 0 and 1.

If probability $\geq 0.5 \rightarrow$ Predict Churn

Else $\rightarrow$ Predict No Churn

Training

The model was trained using:

`model.fit(X_train, y_train)`

The algorithm estimates coefficients that maximize likelihood of correct classification.

6. Prediction and Evaluation

Predictions were made on test data:

```
y_pred = model.predict(X_test)
```

Accuracy

Accuracy = 0.8

This means 80% of test samples were correctly classified.

Total test samples = 40

Correct predictions = 32

Confusion Matrix

```
[ 32  1
   7  0]
```

Interpretation:

- True Negatives (TN) = 32
- False Positives (FP) = 1
- False Negatives (FN) = 7
- True Positives (TP) = 0

The model correctly identifies non-churn customers but fails to identify churn customers.

Classification Report

Class Precision Recall F1-score Support

```
0    0.82    0.97    0.89    33
1    0.00    0.00    0.00     7
```

Class	Precision	Recall	F1-Score	Support
0	0.82	0.97	0.89	33
1	0.00	0.00	0.00	7

Overall Accuracy = 0.80

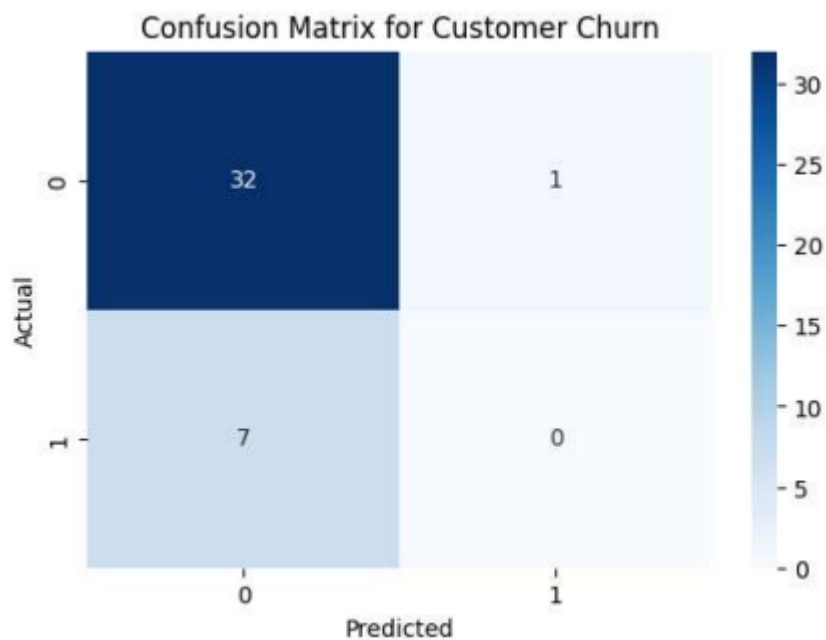
Macro Average F1-score = 0.44

Weighted Average F1-score = 0.73

8. Graphical Evaluation

A confusion matrix heatmap was plotted using seaborn:

- Blue color intensity represents prediction counts.
- Majority predictions are concentrated in True Negative cell.
- No True Positive cases were detected.



This visually confirms class imbalance and poor churn detection.